

LeWorldModel

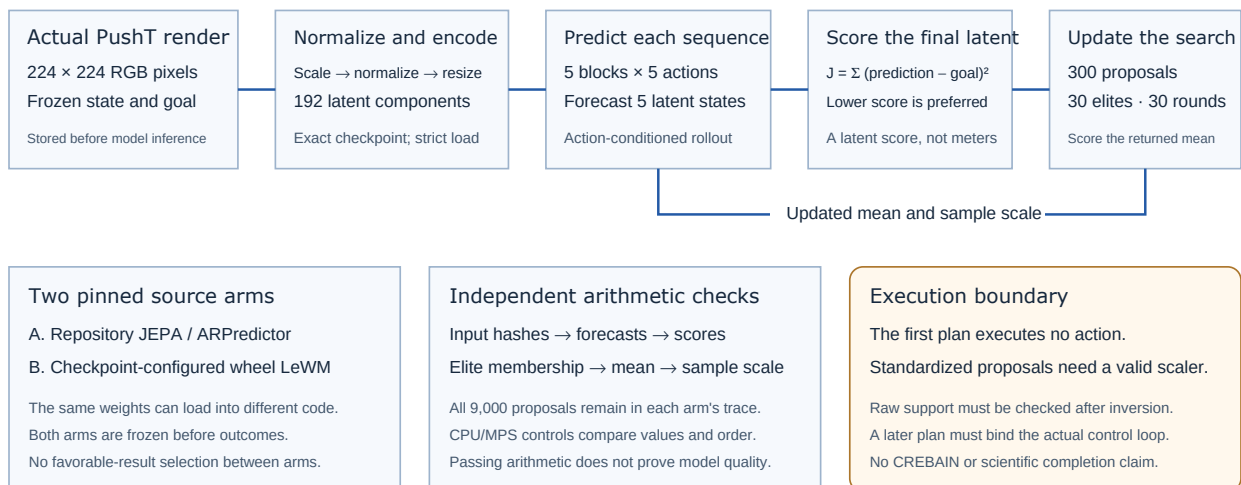
From pixels to a scored action

A Prisoma mathematics and evidence guide

The local engineering adapter computes pretrained forecasts and complete candidate searches. It executes no raw action and establishes no scientific result.

From pixels to a scored action sequence

Pinned LeWorldModel · local M4 engineering candidate · scientific gates remain open



Solid arrows: actual inference/search data flow. This diagram shows the bounded probe, not the future experiment authority graph.

Read the symbols before reading the score

Pixels, arena coordinates, standardized actions, and learned latent vectors are different quantities. The following pages state their units and show each calculation with small examples.

The SVG and PDF use vector geometry and text. Zoom to inspect the model path and its evidence boundary.

Canonical source: docs/lewm/MATHEMATICS.md

Source SHA-256: fc91f87a9b126bda11652b79d62ec7bcfe794deb86acc94d3edb80ca9f854513

This note describes the frozen engineering probe. It does not report model quality or physical accuracy.

State, observation, and action

The PushT owner has a physics state, denoted by s_t . It includes more information than the seven values returned by `_get_obs`. For example, the returned vector omits the block's velocity and contact-solver memory.

The observation o_t is an RGB image with 224 by 224 pixels. Each stored channel value is an integer from 0 through 255. The encoder receives the normalized image x_t .

For channel c , the preprocessing operation is:

```
x_t[c] = (o_t[c] / 255 - mean[c]) / std[c]
mean = [0.485, 0.456, 0.406]
std = [0.229, 0.224, 0.225]
```

The values are dimensionless. The released operation order applies normalization before resizing. The first engineering arm already renders at 224 pixels, but retains the same operation order.

A raw action u_t has two components. PushT declares each component within $[-1, 1]$. In relative mode, the controller target is the current agent position plus $100 \cdot u_t$. The position coordinates are simulator arena units. They are not meters.

The controller runs ten physics steps for each action. Each physics step lasts 0.01 simulated seconds. Thus, one raw action spans 0.1 simulated seconds.

The predictor consumes a standardized action a_t :

```
a_t[j] = (u_t[j] - action_mean[j]) / action_scale[j]
u_t[j] = a_t[j] * action_scale[j] + action_mean[j]
```

Here, j selects one of the two action components. The action mean and scale must come from a declared, content-bound dataset fit. The frozen first probe has no such fit. Therefore, it cannot execute its standardized recommendations.

A model action block concatenates five consecutive two-dimensional actions. Its width is ten. It represents 0.5 simulated seconds when an authorized scaler and execution path exist. Five blocks represent 2.5 simulated seconds.

Encoding and prediction

The encoder maps an image to a 192-component latent vector:

```
z_t = encoder_and_projector(x_t)
```

Each latent component is a learned, dimensionless quantity. It is not a position, velocity, probability, or confidence interval.

The action encoder maps an action block to its learned embedding. The autoregressive predictor combines recent latent states with the corresponding action embeddings. It uses at most three model time points as context.

With one observed frame and five action blocks, a rollout returns the observed latent and five predicted latents. The captured array therefore has six latent time points. The two frozen source arms preserve their own actual rollout implementation.

Goal score

Let g be the encoded goal image. Let z_{hat}_i be the final latent predicted for candidate i . The released objective is:

$$J_i = \text{sum over } d=1..192 \text{ of } (z_{\text{hat}}_i[d] - g[d])^2$$

The implementation calls an elementwise mean-squared-error operation, then sums its components. Thus, its final score is a sum of squared errors, rather than their mean. Smaller scores receive preference. These scores are not physical goal distances.

For a two-component illustration, let the prediction be $[1, 2]$ and the goal be $[0, 1]$. The score is $(1 - 0)^2 + (2 - 1)^2 = 2$. This illustration is not a measured model result.

Cross-entropy method

The search uses 30 rounds. Each round proposes 300 sequences and retains 30 elite sequences. Every sequence has five blocks of width ten.

At round r , the solver samples independent standard-normal values ϵ_i :

```
candidate_i = mu_r + sigma_r * epsilon_i  
candidate_0 = mu_r
```

Multiplication applies separately to each sequence coordinate. The initial mean μ_0 is zero. The initial scale σ_0 is one. The code calls this scale `var`, although the update computes a standard deviation.

Let E_r contain the 30 candidates with the smallest scores. For each coordinate:

```
mu_(r+1) = sum over i in E_r of candidate_i / 30  
sigma_(r+1) = sqrt(sum over i in E_r of (candidate_i - mu_(r+1))^2 / 29)
```

The denominator is 29 because the released code uses the sample standard deviation. For an illustrative two-member elite set $[1, 3]$, the mean is 2 and this standard deviation is $\sqrt{2}$. A population-standard-deviation update would produce 1. The verifier rejects that substitution.

The solver does not enforce raw action bounds. A supplied `Box` determines dimensions only. The probe records this limitation. It does not silently clip proposals or rename them supported actions.

After round 30, the solver returns its updated mean. The probe separately predicts and scores that exact mean before recording the recommendation. Every sampled proposal, prediction, cost, elite index, mean, and scale remains available for reconstruction.

Experiment order and present boundary

The current probe records source identity, preprocessing, actual input pixels, and candidate tensors before loading the model. It then records forecasts and planner calculations. It executes no action and obtains no branch outcome label.

A complete Prisoma experiment requires a stronger order:

1. Freeze the environment checkpoint, observations, action support, and candidate roster.
2. Compute each declared model and matched-baseline forecast.
3. Commit forecasts, scores, abstentions, and the selected action.
4. Execute the selected action through the Agent Bridge.
5. Record its execution receipt.
6. Evaluate independent reference branches from the same accepted checkpoint.
7. Record labels and replay the complete comparison.

An environment state vector cannot substitute for a complete checkpoint. An in-process ordering rule cannot prove that arbitrary code never accessed future labels. The exact-fork and future-label authority gates remain separate from this local model execution probe.